

CS-1204

Object-Oriented Programming

(3+1)

Lab

By

Engr. Muhammad Awais

File Handling in C++

Objectives

- Understand the need for file handling.
- Learn about C++ file handling classes.
- Explore file modes.
- Understand file pointers and their usage.
- Practice with examples to strengthen concepts.

Key Points

- Variables store data in temporary memory (RAM).
- After program ends → data is lost.
- To keep data permanently → we use files stored on disk.

Classes for File Handling

- C++ provides three main classes
 - **ifstream** → Input file stream (for reading).
 - **ofstream** → Output file stream (for writing).
 - **fstream** → File stream (for both reading & writing).

```
#include <fstream>
using namespace std;
```

Must include *fstream* header file to use input and output file stream.

```
ifstream fin;    // read
ofstream fout;   // write
fstream file;    // read + write
```

Steps to use a file

- Declare a stream object (ifstream, ofstream, fstream).
- Open the file using constructor.
- Perform read/write operations.
- Close the file with close().

```
ofstream fout("data.txt"); // open file for writing
fout << "Hello File";      // write data
fout.close();              // close file
```

File open modes in C++:

- `ios::out` → Open file for writing.
- `ios::in` → Open file for reading.
- `ios::app` → Append data at the end.
- `ios::trunc` → Delete previous content.

Combining file modes

- Modes can be combined using |

cpp

```
fstream file("test.txt", ios::out | ios::in);
```

- Open file for **both reading & writing**.

cpp

```
ofstream fout("log.txt", ios::app | ios::out);
```

- Open file in **append mode** (previous data preserved).

Open file for writing.

```
source.cpp iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream out("greet.txt");
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

```
source.cpp iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     fstream out("greet.txt", ios::out);
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

```
*] source.cpp iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     fstream out("greet.txt", ios::out | ios::app);
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

```
source.cpp iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream out("greet.txt", ios::out);
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

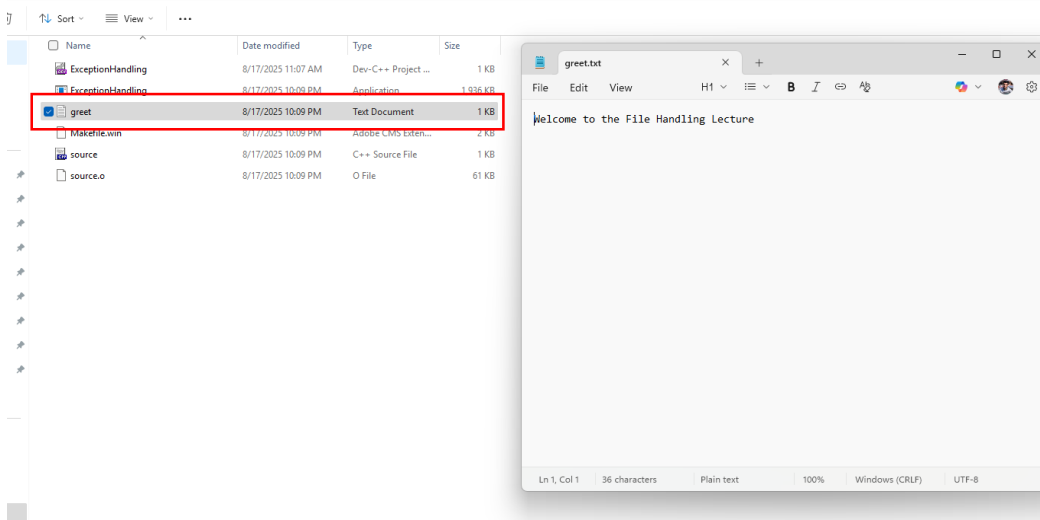
```
source.cpp iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     fstream out("greet.txt", ios::out | ios::trunc);
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

ios::out → Open file for writing.

```
source.cpp | iostream
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     fstream out("greet.txt", ios::out);
6     out << "Welcome to the File Handling Lecture";
7     out.close();
8     return 0;
9 }
```

Opening a file in write mode (ios::out) will:

- Create the file if it doesn't already exist.
- Truncate (delete old content) if the file already exists.
- By default, when you open a file without specifying a truncate or append mode it opens in truncate mode.
- Each time you run the program, the file content will be overridden (old content is erased).

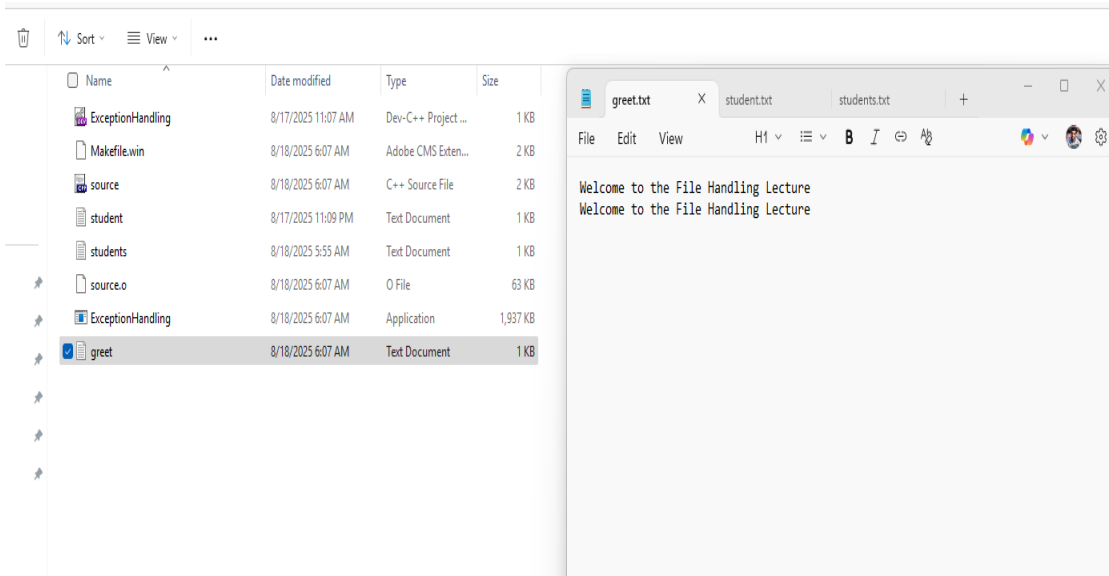


ios::app → Open file for writing in Append Mode.

```
#include<iostream>
#include<fstream>
using namespace std;
int main(){
    fstream out("greet.txt", ios::out | ios::app);
    out << "Welcome to the File Handling Lecture" << endl;
    out.close();
    return 0;
}
```

Opening a file in append mode with (ios::app) will:

- Create the file if it doesn't already exist.
- Start writing from the end of the old content.
- The result on the left shows that the lines are appended at end when we run the program twice.



Writing to a File

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ofstream fout("student.txt");
    fout << "Roll No: 101" << endl;
    fout << "Name: Ali" << endl;
    fout.close();
    cout << "Data written successfully!";
}
```

Similar to, how we use *cout* for writing on a standard console. Here we use file stream object *fout* to write in the files.

Reading from a File

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ifstream fin("student.txt");
    string line;
    while (getline(fin, line)) {
        cout << line << endl;
    }
    fin.close();
}
```

Similar to, how we use *cin* for taking input from the user.

Here we use file stream object *fin* to read line from the file.

File Pointers

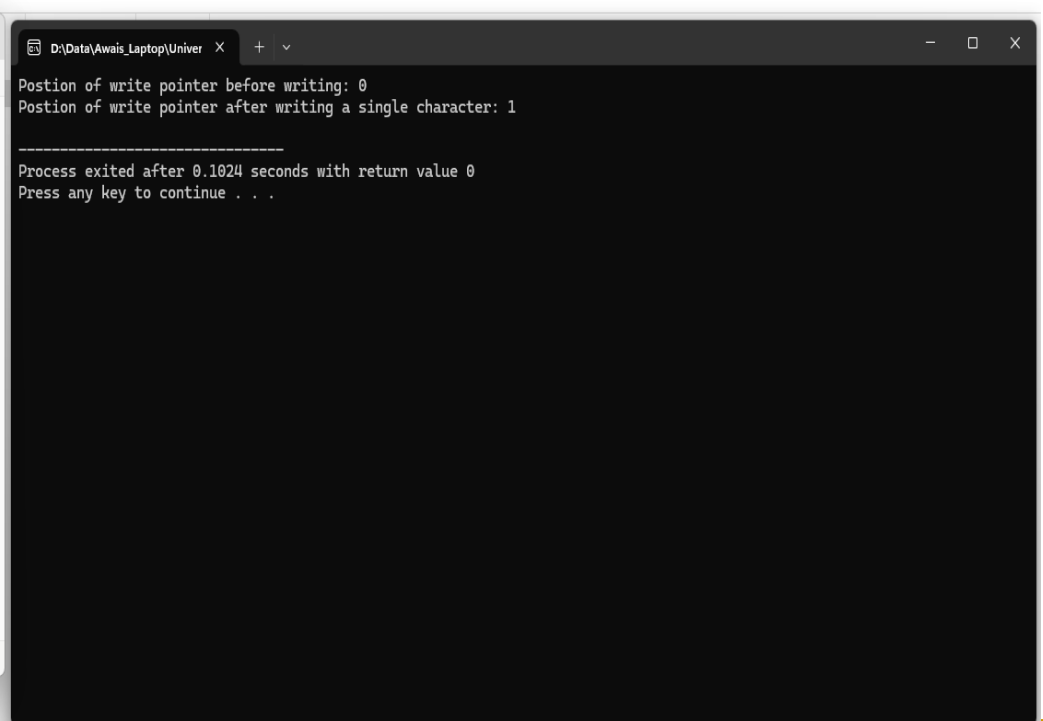
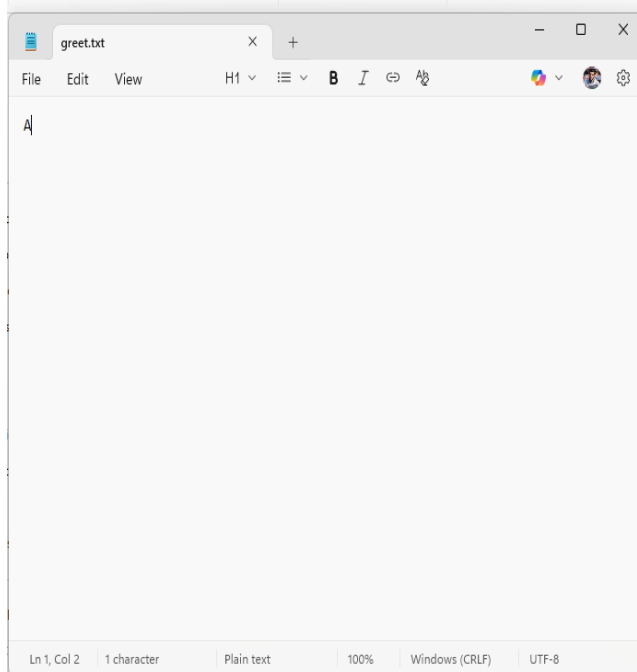
- **tellg()** → Get current get (read) pointer position.
- **tellp()** → Get current put (write) pointer position.
- **seekg(offset, direction)** → Move get pointer.
- **seekp(offset, direction)** → Move put pointer.

Directions:

- ❑ **ios::beg** → Beginning of file.
- ❑ **ios::cur** → Current position.
- ❑ **ios::end** → End of file.

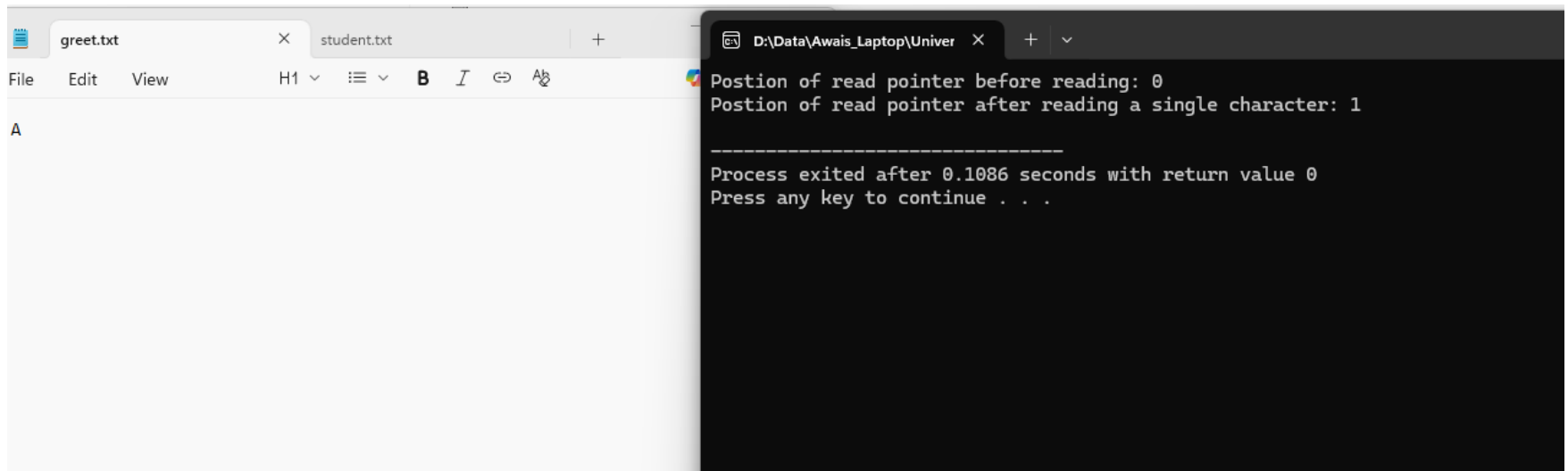
tellp() → Get current put (write) pointer position

```
#include<iostream>
#include<fstream>
using namespace std;
int main(){
    fstream out("greet.txt", ios::out);
    cout << "Postion of write pointer before writing: " << out.tellp() << endl;
    out << "A";
    cout << "Postion of write pointer after writing a single character: " << out.tellp() << endl;
    out.close();
    return 0;
}
```



tellg() → Get current get (read) pointer position.

```
#include<iostream>
#include<fstream>
using namespace std;
int main(){
    fstream in("greet.txt", ios::in);
    cout << "Postion of read pointer before reading: " << in.tellg() << endl;
    char ch;
    in.get(ch);
    cout << "Postion of read pointer after reading a single character: " << in.tellg() << endl;
    in.close();
    return 0;
}
```



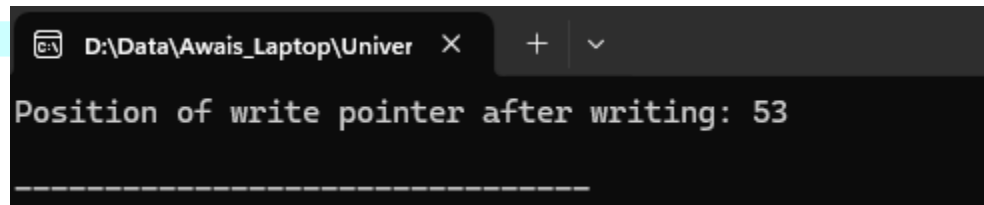
The screenshot shows a code editor with two tabs: 'greet.txt' and 'student.txt'. The 'greet.txt' tab is active, showing a text file with the letter 'A'. To the right, a terminal window titled 'D:\Data\Awais_Laptop\Univer' displays the output of the program. The output shows the initial position of the read pointer at 0, then after reading the character 'A', the position is 1. The terminal also shows the process exiting after 0.1086 seconds with a return value of 0 and a prompt to press any key to continue.

```
Postion of read pointer before reading: 0
Postion of read pointer after reading a single character: 1

-----
Process exited after 0.1086 seconds with return value 0
Press any key to continue . . .
```

seekp(offset, direction) → Move put pointer.

```
#include<iostream>
#include<fstream>
using namespace std;
int main(){
    fstream file("student.txt", ios::in | ios::out | ios::trunc);
    file << "01,Awais,3.5 ";
    file << "02,Kashif,3.8 ";
    file << "03,Zahid,2.5 ";
    file << "04,Ahmed,4.0 ";
    cout << "Position of write pointer after writing: " << file.tellp() << endl;
    //one record takes 13 characters and the moment pointer is at 53 means next writing will be start from
    //53 but lets move it back to override the last line
    //53-13 = 40
    //file.seekg(file.tellp()-13); //absoulte postion without direction.
    file.seekp(-13, ios::cur); //relative with reference to current position.
    file << "05,Nadir,2.78";
    file.close();
    return 0;
}
```



```
D:\Data\Awais_Laptop\Univer x + v
Position of write pointer after writing: 53
-----
```

```
01,Awais,3.5 02,Kashif,3.8 03,Zahid,2.5 05,Nadir,2.78
```

seekg(offset, direction) → Move get pointer.

```
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     fstream file("student.txt", ios::in | ios::out);
6     cout << "Position of pointer before reading: " << file.tellg() << endl;
7     char ch;
8     cout << "Move the read pointer position to 13" << endl;
9     file.seekg(13);
10    while(file.get(ch)){
11        cout << "Character Read: " << ch << " Position of pointer after reading: " << file.tellg() << endl;
12    }
13    file.close();
14    return 0;
15 }
```

01,Awais,3.5 02,Kashif,3.8 03,Zahid,2.5 05,Nadir,2.78

```
D:\Data\Awais_Laptop\Univer X + v
Position of pointer before reading: 0
Move the read pointer position to 13
Character Read: 0 Position of pointer after reading: 14
Character Read: 2 Position of pointer after reading: 15
Character Read: , Position of pointer after reading: 16
Character Read: K Position of pointer after reading: 17
Character Read: a Position of pointer after reading: 18
Character Read: s Position of pointer after reading: 19
Character Read: h Position of pointer after reading: 20
Character Read: i Position of pointer after reading: 21
Character Read: f Position of pointer after reading: 22
Character Read: , Position of pointer after reading: 23
Character Read: 3 Position of pointer after reading: 24
Character Read: . Position of pointer after reading: 25
Character Read: 8 Position of pointer after reading: 26
Character Read: Position of pointer after reading: 27
Character Read: 0 Position of pointer after reading: 28
Character Read: 3 Position of pointer after reading: 29
Character Read: , Position of pointer after reading: 30
Character Read: Z Position of pointer after reading: 31
Character Read: a Position of pointer after reading: 32
Character Read: h Position of pointer after reading: 33
Character Read: i Position of pointer after reading: 34
Character Read: d Position of pointer after reading: 35
Character Read: , Position of pointer after reading: 36
Character Read: 2 Position of pointer after reading: 37
Character Read: . Position of pointer after reading: 38
Character Read: 5 Position of pointer after reading: 39
Character Read: Position of pointer after reading: 40
Character Read: 0 Position of pointer after reading: 41
Character Read: 5 Position of pointer after reading: 42
Character Read: , Position of pointer after reading: 43
Character Read: N Position of pointer after reading: 44
Character Read: a Position of pointer after reading: 45
Character Read: d Position of pointer after reading: 46
Character Read: i Position of pointer after reading: 47
Character Read: r Position of pointer after reading: 48
Character Read: , Position of pointer after reading: 49
Character Read: 2 Position of pointer after reading: 50
Character Read: . Position of pointer after reading: 51
Character Read: 7 Position of pointer after reading: 52
Character Read: 8 Position of pointer after reading: 53

-----
Process exited after 0.1124 seconds with return value 0
Press any key to continue . . .
```

References

- <https://www.geeksforgeeks.org/cpp/file-handling-c-classes/>
- https://www.w3schools.com/cpp/ref_fstream_fstream.asp

Quiz

CLO1->C3

- What type of polymorphism is used in the code snippet given below?

```
#include <iostream>
using namespace std;

class Print {
public:
    void show(int x) { cout << x; }
    void show(double y) { cout << y; }
};
```

Quiz

CLO2->C3

Question 02-:

Create a base class: **Employee** with a virtual function **calculateSalary()**.

Derive two classes: **FullTime** and **PartTime**.

Each class should override the function with its own salary calculation formula:

FullTime Employee:

$$\text{Salary} = \text{Basic Pay} + (\text{Basic Pay} \times \text{Allowance \%}) - (\text{Basic Pay} \times \text{Tax \%})$$

Parttime Employee:

$$\text{Salary} = \text{Hours Worked} \times \text{Rate per Hour}$$

Use a base class pointer to call **calculateSalary()** for both employee types and display the salaries on the output.

Practice Example

Task 1: Create, Write, Read, and Append a File

- Create a file named notes.txt.
- Write three different lines of text into the file.
- Open the file in read mode and display all its contents on the screen.
- Append your name and roll number to the file without overwriting existing content.

Practice Example

Task 2: Count Number of Lines in a File

- Open a file named notes.txt in read mode.
- Count the total number of lines present in the file.
- Display the total line count on the screen.

Task 3: Copy Content from One File to Another

- Open a file named notes.txt in read mode.
- Count the total number of lines present in the file.
- Display the total line count on the screen.

Practice Example

Task 4: Write Student Details and Then Read Them

- Create a file named students.txt.
- Write the name and roll number of three students in the file.
- Open the file in read mode and display all student details on the screen.